PHP:

PHP (recursive acronym for PHP: Hypertext Preprocessor) is a widely-used open source general-purpose scripting language that is especially suited for web development and can be embedded into HTML. It allows web developers to create dynamic content that interacts with databases. PHP is basically used for developing web based software applications.

PHP is mainly focused on server-side scripting, so you can do anything any other CGI(Common Gateway Interface) program can do, such as collect form data, generate dynamic page content, or send and receive cookies. Code is executed in servers, that is why you'll have to install a sever-like environment enabled by programs like XAMPP which is an Apache distribution.

Where is PHP used? (Applications of PHP)

Three are the main areas where PHP scripts are used:

Server-side scripting

This is the most used and main target for PHP. You need three things to make this work the way you need it. The PHP parser , a web server and a web browser.

Command line scripting

You can make a PHP script to run it without any server or browser. You only need the PHP parser to use it this way. This type of usage is ideal for scripts regularly executed using cron (on Linux) or Task Scheduler (on Windows). These scripts can also be used for simple text processing tasks.

Writing desktop applications

PHP may not the very best language to create a desktop application with a graphical user interface, but if you know PHP very well, and would like to use some advanced PHP features in your client-side applications you can also use PHP-GTK to write such programs. You also have the ability to write cross-platform applications this way.

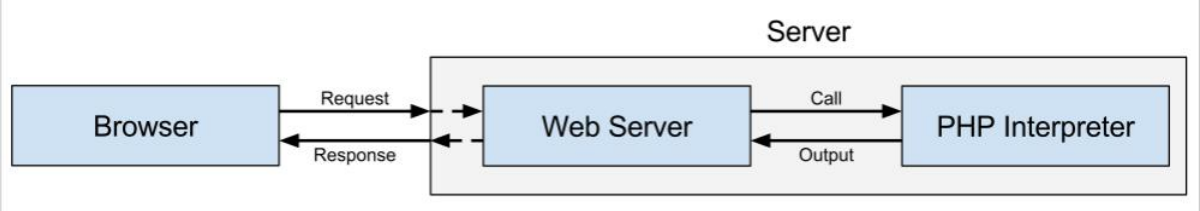
In our scenario, we will mostly look at the server-side scripting using PHP.

Features of PHP

- **Open Source:** PHP is free to use, distribute, and modify, fostering a large and active community.
- **Server-Side Scripting:** Primarily designed for server-side web development, handling dynamic content generation.
- **Cross-Platform Compatibility:** Runs on various operating systems (Windows, Linux, macOS) and web servers (Apache, Nginx, IIS).
- **Embeddable in HTML:** PHP code can be directly embedded within HTML pages, making it easy to integrate dynamic functionality.
- **Large and Active Community:** A vast community provides extensive documentation, tutorials, libraries, and frameworks.
- **Object-Oriented Programming (OOP):** Supports OOP concepts like classes, objects, inheritance, polymorphism, and encapsulation.
- **Database Connectivity:** Offers excellent support for various databases, including MySQL, PostgreSQL, Oracle, and more.
- **Extensive Library Support:** A rich collection of built-in functions and extensions simplifies common tasks like file handling, image manipulation, and data processing.
- **Framework Availability:** Numerous powerful frameworks (e.g., Laravel, Symfony, CodeIgniter) streamline development and promote best practices.
- **Scalability:** Can handle small to large-scale web applications with proper architecture and optimization.
- **Relatively Easy to Learn:** The syntax is generally considered easier to grasp compared to some other server-side languages.
- **Fast Execution:** PHP is generally fast, especially with opcode caching mechanisms.
- **Support for Various Protocols:** Supports HTTP, FTP, SMTP, and other common internet protocols.
- **Dynamic Content Generation:** Enables the creation of interactive and personalized web pages based on user input, database information, and other factors.
- **Support for Web Services:** Can be used to create and consume web services (SOAP and RESTful).
- **Garbage Collection:** Automatic memory management simplifies development and reduces memory leaks.
- **Security Features:** Offers features and best practices to build secure web applications, although developers need to be mindful of security vulnerabilities.

How PHP script work

- PHP always start with a browser making a request for a web page.
- This request is going to hit the web server.
- The web server will then analyze it and determine what to do with it.
- If the web server determines that the request is for a PHP file (often index.php), it'll pass that file to the PHP interpreter.
- The PHP interpreter will read the PHP file, parse it (and other included files) and then execute it.
- Once the PHP interpreter finishes executing the PHP file, it'll return an output.
- The web server will take that output and send it back as a response to the browser.



PHP First program.

```
<!DOCTYPE html>
<html>
	<body>
		<?php
		echo "My first PHP script!";
		?>
	</body>
</html>
```

We can omit the html tags and can directly start the php script.

```
<?php
	echo "My first PHP script!";
?>
```

Running PHP Scripts

Run Steps with XAMPP:

1. **Install XAMPP (If not already installed before):**

i. Download the appropriate XAMPP installer for your operating system (Windows, macOS, or Linux) from the official Apache Friends website (https://www.apachefriends.org/download.html).
ii. Run the installer and follow the on-screen instructions. You'll be asked to choose components to install; make sure at least **Apache** and **PHP** are selected. You can also select MySQL (MariaDB) if you anticipate needing a database later.
iii. Choose an installation directory (the default is usually fine).

2. **Start the XAMPP Control Panel:**

   Once the installation is complete, open the XAMPP Control Panel application. You can usually find it in your Start Menu (Windows), Applications folder (macOS), or by searching for "XAMPP Control Panel".

3. **Start the Apache Server:**

   i. In the XAMPP Control Panel, you'll see a list of modules (Apache, MySQL, etc.).
   ii. Click the **"Start"** button next to the **Apache** module.
   iii. If Apache starts successfully, you should see its status change to green, and the "PID" and "Ports" columns will be populated. If it fails to start, there might be a port conflict with another application (like Skype or IIS). You might need to configure Apache to use a different port in that case.

4. **Locate the XAMPP "htdocs" Directory:**

   i. The htdocs directory is the **document root** for the Apache web server in XAMPP. This is where you need to place your PHP files so that Apache can serve them through your web browser.
   ii. The location of the htdocs directory depends on where you installed XAMPP. Common locations are:
   iii. **Windows:** C:¥xampp¥htdocs
   iv. **macOS:** /Applications/XAMPP/xamppfiles/htdocs

5. **Save the PHP File in the "htdocs" Directory (or a Subdirectory):**

   i. Open a text editor.
   ii. Copy and paste the PHP "Hello, World!" code into the editor.
   iii. Save the file as hello.php. **Crucially, save this file inside the** htdocs **directory** (or a subdirectory within it if you want to organize your projects).
   iv. For example: You could save it directly as C:¥xampp¥htdocs¥hello.php (on Windows).

v. Or you could create a new folder inside htdocs called myproject and save it as C:¥xampp¥htdocs¥myproject¥hello.php.

6. **Access the PHP File through Your Web Browser:**

   i. Open your web browser (e.g., Chrome, Firefox, Safari).
   ii. In the address bar, type the URL to access your PHP file. The URL will be based on the location where you saved the file within the htdocs directory:
   iii. **If** hello.php **is directly in** htdocs**:** Type http://localhost/hello.php and press Enter.
   iv. **If** hello.php **is in a subdirectory (e.g.,** myproject**) within** htdocs**:** Type http://localhost/myproject/hello.php and press Enter.

7. **View the Output:**

Basic PHP syntax

- A PHP scripting block always starts with <?php and ends with ?>. A PHP scripting block can be placed anywhere in the document.
- On servers with shorthand support enabled you can start a scripting block with <? and end with ?>.
- For maximum compatibility, it is recommended that you use the standard form (<?php) rather than the shorthand form.
- If a file contains only PHP code, it is preferable to omit the PHP closing tag at the end of the file.
- Each code line in PHP must end with a semicolon. The semicolon is a separator and is used to distinguish one set of instructions from another.
- The file must have a .php extension. If the file has a .html extension, the PHP code will not be executed.

Case Sensitivity in PHP

- PHP is a SEMI-CASE-SENSITIVE language which means some features are case-sensitive while others are not.
- Following are case-insensitive
  - All the keywords (if, else, while, for, foreach, etc.)
  - Functions
  - Statements

- Classes

- PHP Variables are case-sensitive.
  - Only variables with different cases are treated differently.

```
<?php
// Assign value to variable
$color = "blue";
// Try to print variable value
echo "The color of the sky is " . $color . "<br>";
echo "The color of the sky is " . $Color . "<br>";
echo "The color of the sky is " . $COLOR . "<br>";
?>
```

If we try to run the above program, first line will be executed normally and error occurs in the last two lines.

Comments in PHP

In PHP, we use // or hash symbol (#) to make a single-line comment or /* and */ to make a large comment block.

```
<?php
//This is a comment
/*
This is
a comment
block
*/
?>
```

Variables in PHP

Variables in PHP are containers for storing data, which can be used and manipulated throughout the script. PHP is a loosely typed language, meaning you don't need to declare the data type; it is assigned automatically based on the value.

- All variables in PHP are denoted with a leading dollar sign ($).
- PHP Variables are case-sensitive.

PHP is a Loosely Typed Language

- In PHP, a variable does not need to be declared before adding a value to it.

- PHP automatically converts the variable to the correct data type, depending on its value.
- In a strongly typed programming language, you have to declare (define) the type and name of the variable before using it.
- In PHP, the variable is declared automatically when you use it.

Naming Rules for Variables

- A variable name must start with a letter or an underscore "_"
- A variable name can only contain alpha-numeric characters and underscores (a-z, A-Z, 0-9, and _)
- A variable name should not contain spaces. If a variable name is more than one word, it should be separated with an underscore ($my_string), or with capitalization ($myString)

Isset() Function.

The isset() function checks whether a variable is set, which means that it has to be declared and is not NULL.

This function returns true if the variable exists and is not NULL, otherwise it returns false.

Note: If multiple variables are supplied, then this function will return true only if all of the variables are set.

Example

```
<?php
$a = 0;
// True because $a is set
if (isset($a)) {
  echo "Variable 'a' is set.<br>";
}

$b = null;
// False because $b is NULL
if (isset($b)) {
  echo "Variable 'b' is set.";
}
?>
```

Output

```
Variable 'a' is set.
```

PHP Data Types

- Variables can store data of different types, and different data types can do different things.
- Some of the data types are listed below:
    - **Integer:** Represents whole numbers without decimal points.
        - Example: $age = 25;
    - **Float:** Represents numbers with decimal points (floating-point numbers).
        - Example: $price = 19.99;
    - **String:** Represents sequences of characters enclosed in quotes.
        - Example: $name = "John Smith";
    - **Boolean:** Represents a value that is either true or false.
        - Example: $isLogged = true;
    - **Array:** Represents an ordered collection of elements.
        - Example: $numbers = [1, 2, 3, 4, 5];
    - **Object:** Represents an instance of a class.
        - Example: $user = new User();
    - **Null:** Represents the absence of a value.
        - Example: $variable = null;
    - **Resource**:
        - $fp = fopen('file.ext', 'r');
        - var_dump($fp);

PHP Constants

- A constant is simply a name that holds a single value. As its name implies, the value of a constant cannot be changed during the execution of the PHP script. From PHP 7, a constant can hold an array.
- A constant can be accessed from anywhere in the script.
- Use the define() function or const keyword to define a constant.

```
<?php
// Define a constant
define("PI", 3.14);

// Use the constant
$radius = 5;
$area = PI * $radius * $radius;

echo "The area of a circle with radius $radius is: $area";
?>
```

```
<?php
const MY_CONSTANT = 5;
echo MY_CONSTANT;
?>
```

Conditional Statements in PHP

Conditional statements are used to execute different code based on different conditions.

The If statement

The **if** statement executes a piece of code if a condition is true.

Syntax :

```
if (condition) {
// code to be executed in case the condition is true
}
```

Example:

```
<?php
$age = 18;
if ($age < 20) {
echo "You are a teenager";
}
?>
```

The If. . . Else statement

The If. . . Else statement executed a piece of code if a condition is true and another piece of code if the condition is false.

Syntax :

```
if (condition) {
// code to be executed in case the condition is true
}
else {
// code to be executed in case the condition is false
}
```

Example

```
<?php
$age = 25;
if ($age < 20) {
echo "You are a teenager";
}
else {
echo "You are an adult";
}
?>
```

The If. . . Elseif. . . Else statement

This kind of statement is used to define what should be executed in the case when two or more conditions are present.

Syntax

```
if (condition1) {
// code to be executed in case condition1 is true
}
elseif (condition2) {
// code to be executed in case condition2 is true
}
else {
// code to be executed in case all conditions are false
}
```

Example:

```
<?php
$age = 3;
if ($age < 10) {
echo "You are a kid";
} elseif ($age < 20) {
echo "You are a teenager";
} else {
echo "You are an adult";
}
?>
```

Switch Statement

- This statement allows us to execute different blocks of code based on different conditions. Contrary to the if statement, the switch statement is used when the condition contains a particular value rather than a range of values
- Use the switch statement to select one of many blocks of code to be executed.

Syntax

```
switch (value) {
    case value1:
        // code to execute if value matches value1
        break;
    case value2:
        // code to execute if value matches value2
        break;
    // more case statements as needed
    default:
        // code to execute if none of the cases match value
}
```

Example

```
<?php
$message = '';
$role = 'author';
switch ($role) {
        case 'admin':
                $message = 'Welcome, admin!';
                break;
        case 'editor':
        case 'author':
                $message = 'Welcome! Do you want to create a new article?';
                break;
        case 'subscriber':
                $message = 'Welcome! Check out some new articles.';
                break;
        default:
                $message = 'You are not authorized to access this page';
}
echo $message;
?>
```

Loops in PHP

- In PHP, just like any other programming language, loops are used to execute the same code block for a specified number of times.
- Except for the common loop types (for, while, do. . . while), PHP also support **foreach** loops, which is not only specific to PHP.
- Languages like Javascript and C# already use **foreach** loops

The for loop

The for loop is used when the programmer knows in advance how many times the block of code should be executed. This is the most common type of loop encountered in almost every programming language.

Syntax

```
for (initialization; condition; step){
// executable code
}
```

Example

```
for ($i=0; $i < 5; $i++) {
echo "This is loop number $i";
}
```

Result

```
This is loop number 0
This is loop number 1
This is loop number 2
This is loop number 3
This is loop number 4
```

The while loop

The while loop is used when we want to execute a block of code as long as a test expression continues to be true.

```
while (condition){
// executable code
```

```
}
```

Example

```
$i=0; // initialization
while ($i < 5) {
echo "This is loop number $i";
$i++; // step
}
```

Result

```
This is loop number 0
This is loop number 1
This is loop number 2
This is loop number 3
This is loop number 4
```

The do. . . while loop

The do...while loop is used when we want to execute a block of code at least once and then as long as a test expression is true.

Syntax

```
do {
// executable code
}
while (condition);
```

Example

```
$i = 0; // initialization
do {
$i++; // step
echo "This is loop number $i";
}
while ($i < 5);
// condition
```

Output

```
This is loop number 1
This is loop number 2
This is loop number 3
This is loop number 4
This is loop number 5
```

The foreach loop

The foreach loop is used to loop through arrays, using a logic where for each pass, the array element is considered a value and the array pointer is advanced by one, so that the next element can be processed.

Syntax

```
foreach (array as value) {
// executable code
}
```

Example

```
$var = array('a','b','c','d','e');
// array declaration
foreach ($var as $key) {
echo "Element is $key";
}
```

Output

```
Element is a
Element is b
Element is c
Element is d
Element is e
```

PHP Arrays

Arrays are used to store multiple values in a single variable. A simple example of an array in PHP would be:

```
<?php
$languages = array("JS", "PHP", "ASP", "Java");
?>
```

There are three types of arrays in PHP

1. Indexed Arrays
2. Associative Arrays
3. Multidimensional Arrays

Indexed Arrays

We can create these kind of arrays in two ways shown below:

```
<?php
$names = array("Fabio", "Klevi", "John");
?>
```

```
<?php
// this is a rather manual way of doing it
$namesAge = []; // or $namesAge = array();
$names[0] = "Fabio";
$names[1] = "Klevi";
$names[2] = "John";
?>
```

An example where we print values from the array is:

```
<?php
$names = array("Fabio", "Klevi", "John");
echo "My friends are " . $names[0] . ", " . $names[1] . " and " .
$names[2];
?>
```

// RESULT

```
My friends are Fabio, Klevi and John
```

Looping through an indexed array is done like so:

```
<?php
$names = array("Fabio", "Klevi", "John");
$arrayLength = count($names);
for($i = 0; $i < $arrayLength; $i++) {
echo $names[$i];
echo "";
}
?>
```

```
<?php
$names = array("Fabio", "Klevi", "John");
foreach ($names as $key) {
echo "$key";
echo "";
```

```
}
?>
```

Associative Arrays

Associative arrays are arrays which use named keys that you assign. Again, there are two ways we can create them:

```
<?php
$namesAge = array("Fabio"=>"20", "Klevi"=>"16", "John"=>"43");
?>
```

```
<?php
$namesAge = []; // or $namesAge = array();
// this is a rather manual way of doing it
$namesAge['Fabio'] = "20";
$namesAge['Klevi'] = "18";
$namesAge['John'] = "43";
?>
```

An example where we print values from the array is:

```
<?php
$namesAge = array("Fabio"=>"20", "Klevi"=>"16", "John"=>"43");
echo "Fabio's age is " . $namesAge['Fabio'] . " years old.";
?>
```

// RESULT

```
Fabio's age is 20 years old.
```

Looping through an associative array is done like so:

```
<?php
$namesAge = array("Fabio"=>"20", "Klevi"=>"16", "John"=>"43");
foreach($namesAge as $i => $value) {
echo "Key = " . $i . ", Value = " . $value;
echo "";
}
?>
```

Output

```
Key = Fabio, Value = 20
```

```
Key = Klevi, Value = 16
Key = John, Value = 43
```

Multidimensional Arrays

This is a rather advanced PHP stuff, but for the sake of this tutorial, just understand what a multidimensional array is. Basically, it is an arrays the elements of which are other arrays. For example, a three-dimensional array is an array of arrays of arrays. An example of this kind of array would be:

```
<?php
$socialNetowrks = array (
array("Facebook", "feb", 21),
array("Twitter","dec", 2),
array("Instagram","aug", 15));
?>
```

Looping through multidimensional array

```
<?php
$socialNetworks = array(
    array("Facebook", "feb", 21),
    array("Twitter", "dec", 2),
    array("Instagram", "aug", 15)
);

// Loop through the array and display the values
foreach ($socialNetworks as $network) {
    echo "Social Network: " . $network[0] . "<br>";
    echo "Month: " . $network[1] . "<br>";
    echo "Day: " . $network[2] . "<br><br>";
}
?>
```

Output

```
Social Network: Facebook
Month: feb
Day: 21

Social Network: Twitter
Month: dec
Day: 2

Social Network: Instagram
```

```
Month: aug
Day: 15
```

PHP Functions

Functions are a type of procedure or routine that gets executed whenever some other code block calls it.

Apart from its own functions, PHP also let's you create your own functions. The basic syntax of a function that we create is:

```
<?php
function functionName($argument1, $argument2...) {
// code to be executed
}
functionName($argument1, $argument2...); // function call
?>
```

Every function needs a name, optionally has one or more arguments and most importantly, defines some kind of procedure to be followed within the body, that is, code to be executed. Let's see some basic functions:

```
<?php
function showGreeting() { // function definition
echo "Hello Chloe!";
// what this function does
}
showGreeting(); // function call
?>
```

This function would just print the message we wrote:

```
Hello Chloe!
```

Another example would be a function with arguments:

```
<?php
function greetPerson($name) { // function definition with arguments
echo "Hi there, ".$name;
// what this function does
}
greetPerson("Fabio"); // function call
greetPerson("Michael");
?>
```

This time we'd have:

```
Hi there, Fabio
Hi there, Michael
```

More than one argument can be used whenever needed:

```
<?php
function personProfile($name, $city, $job) { // function definition with
arguments
echo "This person is ".$name." from ".$city.".";
echo "";
echo "His/Her job is ".$job.".";
}
personProfile("Fabio", "Tirana", "Web Dev");
echo "";
personProfile("Michael", "Athens", "Graphic Designer");
echo "";
personProfile("Xena", "London", "Tailor");
?>
```

The result would include the printed message together with the arguments:

```
This person is Fabio from Tirana.
His/Her job is Web Dev.
This person is Michael from Athens.
His/Her job is Graphic Designer.
This person is Xena from London.
His/Her job is Tailor.
```

In PHP, just like in many other languages, we can tell functions to return a value upon executing the code. Such example would

be:

```
<?php
function difference($a, $b) { // function definition with arguments
$c = $a - $b;
return $c;
}
echo "The difference of the given numbers is: ".difference(8, 3);
?>
```

The result would be:

```
The difference of the given numbers is: 5
```

Classes and Objects

- In the context of OO software, an object can be almost any item or concept—a physical object such as a desk or a customer; or a conceptual object that exists only in software, such as a text input area , a file, a bank account ,etc.
- An object has -
    - properties (or attributes) that represent the characteristics of the object (e.g: account number and balance of a bank account)
    - behaviors (or operations) that represent the functionalities and the actions related to the object (e.g: deposit and withdraw related with bank account)
- An object holds its state in variables that are often referred to as properties. An object also exposes its behavior via functions (or methods).
- A class is the blueprint of objects. A class defines how an object will be. Objects are created as per the class definitions. So, a class is a template for objects, and an object is an instance of a class.
- Class is considered as logical entity and object is considered as physical entity.

Let's now define a new clas in PHP.

```
<?php
class myClass {
// use 'class' followed by the name of the class
var $var1;
// declared an undefined variable
var $var2 = 5;
// declared a number defined variable
var $var3 = "string"; // declared a string defined variable
function myFunction ($argument1, $argument2) {
// function definition
// function code here
}
}
?>
```

But that is just how the syntax looks like. Below, we give an example of a real-world class, for example class Vehicle:

```
<?php
class Vehicle {
var $brand;

var $speed = 80;
function setSpeed($speedValue) {
$this->speed = $speedValue;
}

function setBrand($brandName) {
$this->brand = $brandName;
}

function printDetails(){
echo "Vehicle brand is: ".$this->brand;
echo "";
echo "Vehicle speed is: ".$this->speed;
}
}

$myCar = new Vehicle;
$myCar->setBrand("Audi");
$myCar->setSpeed(120);
$myCar->printDetails();
?>
```

The result of this code would be:

```
Vehicle brand is: Audi
Vehicle speed is: 120
```

Constructor in PHP classes.

In PHP, a constructor is a special method within a class that is automatically called when a new object (instance) of that class is created. Its primary purpose is to initialize the object's properties (attributes) with default values or values provided during object creation.

Syntax

```
__construct()
```

Example

```
<?php
class MyClass {
```

```
    public $property1;
    public $property2;

    public function __construct($value1, $value2) {
        $this->property1 = $value1;
        $this->property2 = $value2;
        echo "Constructor of MyClass called!<br>";
    }

    public function displayProperties() {
        echo "Property 1: " . $this->property1 . "<br>";
        echo "Property 2: " . $this->property2 . "<br>";
    }
}

// Creating a new object of MyClass and passing values to the constructor
$obj = new MyClass("Hello", 123);
$obj->displayProperties();

?>
```

PHP Forms

PHP forms are a fundamental way to collect user input on websites and process that data on the server. Forms allow users to enter information (like text, selections, file uploads) that can be sent to a server for processing. PHP is particularly well-suited for handling form data because it can easily access and process the submitted information.

Syntax

A basic PHP form has two main parts:

- The HTML form (client-side)
- The PHP processor (server-side)

HTML Form Structure:

```
<form method="POST|GET" action="processing_script.php">
    <!-- Form elements go here -->
    <input type="submit" value="Submit">
</form>
```

PHP Processor Structure:

```
<?php if($_SERVER['REQUEST_METHOD'] == 'POST'/'GET') {
    // Process form data
 }
?>
```

NOTE: We do not specifically need to create HTML form and PHP file separately. We can combine HTML with PHP in a single file as well.

Example

HTML Form (index.html):

```
<form method="post" action="welcome.php">
    Name: <input type="text" name="name"><br>
    Email: <input type="text" name="email"><br>
    <input type="submit" value="Submit">
</form>
```

PHP Processor (welcome.php):

```
<?php if($_SERVER['REQUEST_METHOD'] == 'POST') {
    $name = $_POST['name'];
    $email = $_POST['email'];
    echo "Welcome $name! Your email is $email.";}
?>
```

Accessing Form Elements

Accessing form elements in PHP refers to retrieving the values submitted through HTML form controls (like text inputs, checkboxes, radio buttons, etc.) in your PHP script. PHP provides superglobal arrays ($_GET, $_POST, $_REQUEST) to access these values.

Syntax

For POST method forms: `$_POST['field_name']`
For GET method forms: `$_GET['field_name']`
For either method: `$_REQUEST['field_name']`

Example

HTML Form:

```
<form method="post" action="process.php">
    Username: <input type="text" name="username"><br>
    Remember me: <input type="checkbox" name="remember" value="yes"><br>
    Gender:
    <input type="radio" name="gender" value="male"> Male
    <input type="radio" name="gender" value="female"> Female<br>
```

```
    <select name="country">
        <option value="us">United States</option>
        <option value="uk">United Kingdom</option>
    </select><br>
    <input type="submit" value="Submit">
</form>
```

PHP Processor (process.php):

```
<?php
$username = $_POST['username'];
$remember = isset($_POST['remember']) ? $_POST['remember'] : 'no';
$gender = $_POST['gender'];
$country = $_POST['country'];

echo "Username: $username<br>";
echo "Remember me: $remember<br>";
echo "Gender: $gender<br>";
echo "Country: $country<br>";
?>
```

Instead of using different files, you can use the single php file to contain the ui as well as php code.

Example for GET

```
<?php

class Person {
    public $name;
    public $email;

    // Constructor to initialize the properties
    public function __construct($name, $email) {
        $this->name = $name;
        $this->email = $email;
    }
}
$people = [
    new Person('John Doe', 'john.doe@example.com'),
    new Person('Jane Smith', 'jane.smith@example.com'),
    new Person('Emily Davis', 'emily.davis@example.com'),
    new Person('Michael Brown', 'michael.brown@example.com')
];

// Check if a search parameter is provided
if (isset($_GET['search'])) {
```

```
    $searchTerm = $_GET['search'];
    $searchResults = [];

    // Loop through the people array to search for the term in name
    foreach ($people as $person) {
        if (stripos($person->name, $searchTerm) !== false) {
            $searchResults[] = $person;
        }
    }
    // Return search results
    if (count($searchResults) > 0) {
        echo "<h2>Search Results:</h2>";
        foreach ($searchResults as $person) {
            echo "<p>Name: " . $person->name . "<br>Email: " . $person-
>email . "</p>";
        }
    } else {
        echo "<p>No results found for '$searchTerm'</p>";
    }
} else {
    echo "<p>Please enter a search term.</p>";
}

?>
<form action="getmethod.php" method="GET">
        <label for="search">Enter Name or Email to Search:</label>
        <input type="text" name="search" id="search">
        <button type="submit">Search</button>
</form>
```

Form Validation

Form validation is the process of ensuring that user-submitted data meets certain criteria before processing.

This includes:

- Required field checks
- Data format validation (email, phone numbers)
- Data type validation
- Sanitization to prevent security issues
- Validation should be done on both client-side (for user experience) and server-side (for security).

These validations can be done at the client side before sending data to the server or at the server side when the data is received from the client. The client-side validation is important because of better user experience, while the server side validation should ensure that the invalid data does not enter the system.

Importance of client-side Validation

- Better User experiences
- Since the validation happens in the client's browser, the Response is faster and immediate.
- Saves Precious server resources bandwidth by Minimizing the Server round trips
- Since there is no HTTP Request/response round trip involved in the validation, It saves bandwidth & Server resources

Importance of server-side Validation

The client-side validation gives a better user experience, but not trustworthy. It may fail for many no of reasons. For Example.

- The Javascript may be disabled in the end users browser.
- The Malicious user can send the data directly to the user, without using the application or use some interceptor modify the HTTP Request.
- An Error in Javascript code may result in validation to succeed although data is invalid.

Hence, it is important to validate the data completely at the server side, even though you have done those validations at the client side.

The client side validation is done in client side by using html validation tags and/or using javascript. Here we mainly focus on server side form validation.

Syntax

Common validation functions:

`empty()` - Check if a field is empty

`filter_var()` - Validate/sanitize data

`preg_match()` - Regular expression validation

`strlen()` - Check string length

Example (validation.php)

```
<!DOCTYPE html>
<html >
<head>
    <meta charset="UTF-8">
    <title>PHP Form with Enhanced Validation</title>
</head>
<body>
```

```
<h2>Contact Form</h2>

<form method="POST" action="validation.php">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" ><br><br>

    <label for="address">Address:</label>
    <textarea id="address" name="address"><br><br>

    <label for="email">Email:</label>
    <input type="text" id="email" name="email" ><br><br>

    <input type="submit" value="Submit">
</form>

<?php
// Server-side form validation and processing
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Define variables and initialize with empty values
    $name = $address = $email = "";
    $nameErr = $addressErr = $emailErr = "";

    // Validate Name
    if (empty($_POST["name"])) {
        $nameErr = "Name is required.";
    } else {
        $name = test_input($_POST["name"]);
        // Check if name has only letters and spaces and is at least 2
characters long
        if (!preg_match("/^[a-zA-Z-' ]*$/", $name)) {
            $nameErr = "Only letters and white space allowed in Name.";
        } elseif (strlen($name) < 2) {
            $nameErr = "Name should be at least 2 characters long.";
        }
    }

    // Validate Address
    if (empty($_POST["address"])) {
        $addressErr = "Address is required.";
    } else {
        $address = test_input($_POST["address"]);
        if (strlen($address) < 5) {
            $addressErr = "Address should be at least 5 characters long.";
        }
    }

    // Validate Email
    if (empty($_POST["email"])) {
        $emailErr = "Email is required.";
```

```
    } else {
        $email = test_input($_POST["email"]);
        // Validate email format
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            $emailErr = "Invalid email format.";
        }
    }

    // Display errors or success message
    if (empty($nameErr) && empty($addressErr) && empty($emailErr)) {
        echo "<h3>Form submitted successfully!</h3>";
        echo "<p>Name: $name</p>";
        echo "<p>Address: $address</p>";
        echo "<p>Email: $email</p>";
    } else {
        echo "<h3>There were errors in your form:</h3>";
        echo "<p>$nameErr</p>";
        echo "<p>$addressErr</p>";
        echo "<p>$emailErr</p>";
    }
}

// Function to sanitize input data
function test_input($data) {
    $data = trim($data);  // Remove unnecessary spaces
    $data = stripslashes($data);  // Remove backslashes
    $data = htmlspecialchars($data);  // Convert special characters to
HTML entities
    return $data;
}
?>

</body>
</html>
```

If you run the above program by providing all the valid inputs, the input fields are cleared and provided values are shown at the below.

If you want to retain the values, you need to set the posted values in value field of input field like following.

```
<form method="POST" action="validation.php">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" value="<?php echo
isset($_POST['name']) ? $_POST['name'] : ''; ?>"><br><br>

    <label for="address">Address:</label>
```

```
    <textarea id="address" name="address"><?php echo
isset($_POST['address']) ? $_POST['address'] : ''; ?></textarea><br><br>

    <label for="email">Email:</label>
    <input type="text" id="email" name="email" value="<?php echo
isset($_POST['email']) ? $_POST['email'] : ''; ?>"><br><br>

    <input type="submit" value="Submit">
</form>
```

Cookies and Sessions

Cookies

Cookies are small strings of data that are stored directly in the browser. They are a part of the HTTP protocol. A cookie is a small piece of data that is stored on the user's computer by the web browser. Cookies are often used to store user preferences or other data that needs to persist across multiple sessions or pages on a website.

Creating Cookies

```
<?php

setcookie(cookie_name, cookie_value, [expiry_time], [cookie_path],
[domain], [secure], [httponly]);

?>
```

Example

```
<?php

Setcookie("username", "abc", time() + 60 * 60);

?>
```

Retreiving Cookies data

- To retrieve cookies data in PHP use $_COOKIES.
- To check if it is set or not, use isset() function.

```
<?php
Setcookie("username", "abc", time() + 60 * 60);
?>
<html>
<body>
<?php
	if(isset($_COOKIE["username"])) {
		echo "Cookie is set";
	}
	else {
		echo "Cookie is not set";
	}
?>
</body>
</html>
```

Updating Cookies

For updating cookies only, we need to change the argument by calling setcookie() function. We change name “abc” to “xyz”.

```
<?php
setcookie("username", "xyz", time() + 60 * 60);
?>
```

Deleting Cookies

For deleting cookies we need to set expiry time in negative.

```
<?php

Setcookie("username", "xyz", time() - 3600);

?>
```

Session:

A session in PHP is a way to store information (like user login data) to be used across multiple pages on a website. Unlike cookies, the data is stored on the server, not on the user's browser, which makes it more secure.

HTTP (the protocol web browsers use) is stateless, which means it doesn’t remember anything about the user between different page requests.

A session helps by allowing us to remember the user from page to page — like when someone logs in and navigates around the site without needing to log in again on every page.

It is more secure than cookies.

Session is size independent to store as many data the user wants.

How a PHP Session Works

1. **Starting a session**: When session_start() is called, PHP checks if the user already has a session ID (usually stored in a cookie called PHPSESSID).
2. **If not**, it generates a new unique session ID.
3. **PHP creates a file on the server** to store session data associated with that session ID.
4. As the user moves from page to page, PHP keeps using that session ID to retrieve and store data.

Creation of Session:

In PHP, session starts from session_start() function and data can be set and get by using global variables $_SESSION.

```
<?php
        session_start();
        $SESSION["username"] = "abc";
        $SESSION["userid"] = "1";
?>
<html>
<body>
        <?php
        echo "Session variable is set";
        ?>
<button type="submit" href="logout.html"></button>
</body>
</html>
```

Retrieve session information to another pages

```
<?php

echo "Username:" . " " . $username;
echo "Userid:" . " " . $userid;

?>
```

Update Session Variable

```
$_SESSION["userid"]="1024";
```

Delete/Destroy Session

To destroy session in PHP, first unset all the session variable using session_unset() and call session_destroy() methods.

```
<?php
session_start();
?>
<html>
<body>
<?php

session_unset();
session_destroy();
?>
</body>
</html>
```

Working With PHP and MYSQL

Database Connection
A database connection is a link between a website and a database. This connection allows the website to access and retrieve data from the database, making it possible to store and retrieve information such as user accounts, blog posts, or product listings.

Ways of Connecting to MySQL through PHP
In order to store or access the data inside a MySQL database, you first need to connect to the MySQL database server. PHP offers two different ways to connect to MySQL server:

- MySQLi (Improved MySQL)
- PDO (PHP Data Objects) extensions.

While the PDO extension is more portable and supports more than twelve different databases, MySQLi extension as the name suggests supports MySQL database only.

Syntax: MySQLi

```
$mysqli = mysqli_connect("hostname", "username", "password", "database");
```

Or

```
$mysqli = new mysqli("hostname", "username", "password", "database");
```

Syntax: PDO

```
$pdo = new PDO("mysql:host=hostname;dbname=database", "username",
"password");
```

NOTE: If you just want to connect to MYSQL without any database, you can skip database and still the connection work.

Example: MySqli

```php
<?php

$servername = "localhost";
$username = "root";
$password = "";
$dbname = "classdb";

// Connection
$conn = mysqli_connect($servername, $username, $password,$dbname);

// Check if connection is Successful or not
if (!$conn) {
die("Connection failed: ". mysqli_connect_error());
}
echo "Connected successfully";
?>
```

Example:PDO

```php
<?php

$servername = "localhost";
$username = "root";
$password = "";

try {
    $conn = new PDO("mysql:host=$servername;dbname=classdb", $username,
$password);

    // Set the PDO error mode to exception
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connected successfully";

} catch(PDOException $e) {
    echo "Connection failed: ". $e->getMessage();
}
?>
```

We can use any approach, but in our scenario we are going to use mysqli. The common functions of mysqli are as follows.

Procedural Function	Description	Object-Oriented Equivalent
mysqli_connect()	Opens a new connection to the database.	new mysqli(host, user, pass, db)
mysqli_query($conn, $sql)	Executes a query on the connection.	$conn->query($sql)

Procedural Function	Description	Object-Oriented Equivalent
mysqli_multi_query($conn, $sql)	Executes one or more queries on the provided connection. The queries must be separated by semicolons (;).	$conn->multi_query($sql)
mysqli_fetch_assoc($result)	Fetches a result row as an associative array.	$result->fetch_assoc()
mysqli_fetch_array($result)	Fetches a result row as an array.	$result->fetch_array()
mysqli_fetch_row($result)	Fetches a result row as a numeric array.	$result->fetch_row()
mysqli_num_rows($result)	Returns the number of rows in a result set.	$result->num_rows
mysqli_num_fields($result)	Returns the number of fields in a result set.	$result->field_count
mysqli_real_escape_string($conn, $string)	Escapes special characters in a string.	$conn->real_escape_string($string)
mysqli_insert_id($conn)	Gets last inserted ID.	$conn->insert_id
mysqli_affected_rows($conn)	Returns number of affected rows.	$conn->affected_rows
mysqli_error($conn)	Returns last error.	$conn->error
mysqli_close($conn)	Closes the connection.	$conn->close()

Creating Database using Mysqli

```
<?php

$servername = "localhost";
$username = "root";
$password = "";
$dbname = "classdb";

// Connection
$conn = mysqli_connect($servername, $username, $password,$dbname);

// Check connection
if ($conn === false) {
    die("ERROR: Could not connect. " . mysqli_connect_error());
}

// Attempt to create database
$sql = "CREATE DATABASE classdb";

if (mysqli_query($conn , $sql)) {
    echo "Database created successfully";
} else {
    echo "ERROR: Could not execute $sql. " . mysqli_error($conn);
```

```
}

// Close connection
mysqli_close($conn);
?>
```

Creating Database using PDO

```
<?php

$servername = "localhost";
$username = "root";
$password = "";
$dbname = "classdb";

try {
    // Connect to MySQL without specifying a database to create one
    $pdo = new PDO("mysql:host=$servername", $username, $password);
    // Set the PDO error mode to exception
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Attempt to create the database
    $sql = "CREATE DATABASE $dbname";
    $pdo->exec($sql);
    echo "Database created successfully";
} catch (PDOException $e) {
    echo "ERROR: Could not execute $sql. " . $e->getMessage();
}

// Close connection
$pdo = null;
?>
```

Creating Table

```
<?php
$link = mysqli_connect("localhost", "root", "", "classdb");

// Check connection
if ($link === false) {
    die("ERROR: Could not connect. " . mysqli_connect_error());
}

// Attempt create table query execution
$sql = "
    CREATE TABLE persons (
        id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
        first_name VARCHAR(30) NOT NULL,
        last_name VARCHAR(30) NOT NULL,
```

```
        email VARCHAR(70) NOT NULL UNIQUE
    )
";

if (mysqli_query($link, $sql)) {
    echo "Table created successfully.";
} else {
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}

// Close connection
mysqli_close($link);
?>
```

Inserting data

```
<?php
// Connect to the database
$link = mysqli_connect("localhost", "root", "", "classdb");

// Check connection
if ($link === false) {
    die("ERROR: Could not connect. " . mysqli_connect_error());
}

// Static values to insert
$first_name = 'John';
$last_name = 'Doe';
$email = 'john.doe@example.com';

// Insert query
$sql = "INSERT INTO persons (first_name, last_name, email)
        VALUES ('$first_name', '$last_name', '$email')";

if (mysqli_query($link, $sql)) {
    echo "Record inserted successfully.";
} else {
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}

// Close connection
mysqli_close($link);
?>
```

Inserting Multiple Data

```
<?php
// Connect to the database
$link = mysqli_connect("localhost", "root", "", "classdb");

// Check connection
if ($link === false) {
    die("ERROR: Could not connect. " . mysqli_connect_error());
}

// Array of records to insert
$records = [
    ['John', 'Doe', 'john.doe@example.com'],
    ['Jane', 'Smith', 'jane.smith@example.com'],
    ['Alice', 'Johnson', 'alice.johnson@example.com'],
    ['Bob', 'Brown', 'bob.brown@example.com']
];

// Build multi-query string
$sql = "";
foreach ($records as $record) {
    $first_name = mysqli_real_escape_string($link, $record[0]);
    $last_name = mysqli_real_escape_string($link, $record[1]);
    $email = mysqli_real_escape_string($link, $record[2]);

    $sql .= "INSERT INTO persons (first_name, last_name, email)
             VALUES ('$first_name', '$last_name', '$email');";
}

// Execute multi-query
if (mysqli_multi_query($link, $sql)) {
    echo "All records inserted successfully.";
} else {
    echo "ERROR: Could not execute multi-query. " . mysqli_error($link);
}

// Close connection
mysqli_close($link);
?>
```

Update Data

```
<?php
// Connect to the database
$link = mysqli_connect("localhost", "root", "", "classdb");

// Check connection
if ($link === false) {
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
```

```
// Static values
$id = 1; // ID of the record to update
$new_first_name = 'Jane';
$new_last_name = 'Smith';
$new_email = 'jane.smith@example.com';

// Update query
$sql = "UPDATE persons
        SET first_name='$new_first_name', last_name='$new_last_name',
email='$new_email'
        WHERE id=$id";

if (mysqli_query($link, $sql)) {
    echo "Record updated successfully.";
} else {
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}

// Close connection
mysqli_close($link);
?>
```

Delete Data

```
<?php
// Connect to the database
$link = mysqli_connect("localhost", "root", "", "classdb");

// Check connection
if ($link === false) {
    die("ERROR: Could not connect. " . mysqli_connect_error());
}

// Static ID of the record to delete
$id = 1;

// Delete query
$sql = "DELETE FROM persons WHERE id=$id";

if (mysqli_query($link, $sql)) {
    echo "Record deleted successfully.";
} else {
    echo "ERROR: Could not able to execute $sql. " . mysqli_error($link);
}

// Close connection
mysqli_close($link);
?>
```

Examples of DB operations using HTML Form and php script.

1. Create form and PHP scripts to insert records.

```
<?php

function connect() {
    $servername = "localhost";
    $username = "root";
    $password = "";
    $dbname = "classdb";

    $mysqli = mysqli_connect($servername, $username, $password, $dbname);
    if (!$mysqli) {
        die("Connection failed: " . mysqli_connect_error());
    }
    return $mysqli;
}

$mysqli = connect();

// Handle Create
if (isset($_POST['create'])) {
    $servername = "localhost";
    $username = "root";
    $password = "";
    $dbname = "classdb";

    $mysqli = mysqli_connect($servername, $username, $password, $dbname);
    if (!$mysqli) {
        die("Connection failed: " . mysqli_connect_error());
    }

    $first_name = $_POST['first_name'];
    $last_name = $_POST['last_name'];
    $email = $_POST['email'];

    $insertquery = "INSERT INTO persons (first_name, last_name, email)
VALUES ('$first_name', '$last_name', '$email')";
    if (mysqli_query($mysqli, $insertquery)) {
        echo "Record Inserted Successfully";
    } else {
        echo "ERROR: Could not able to execute $insertquery. " .
mysqli_error($mysqli);
    }
}
```

```
?>

<!DOCTYPE html>
<html lang="en">
<body>
    <h2>Add Person</h2>
    <form method="POST" action="crud.php">
        <input type="text" name="first_name" placeholder="First Name"
required>
        <input type="text" name="last_name" placeholder="Last Name"
required>
        <input type="email" name="email" placeholder="Email" required>
        <button type="submit" name="create">Add</button>
    </form>
</body>
</html>
```

2. Write a PHP program to create a form. Create an insertdb() method to insert the form vlaues into a database table and display() to display the database table value.

```
<?php

$servername = "localhost";
$username = "root";
$password = "";
$dbname = "classdb";
$displayData = false;
// Connection
$mysqli = mysqli_connect($servername, $username, $password,$dbname);

// Check if connection is Successful or not
if (!$mysqli) {
die("Connection failed: ". mysqli_connect_error());
}

if(isset($_POST['insert'])){
$first_name = $_POST['first_name'];
$last_name = $_POST['last_name'];
$email = $_POST['email'];

insertdb($mysqli, $first_name, $last_name, $email);
} else if (isset($_POST['display'])){
    $displayData = true;
}

// Function to insert data into the database
function insertdb($mysqli, $first_name, $last_name, $email) {
```

```
    $insertquery = "INSERT INTO persons (first_name, last_name, email)
VALUES ('$first_name', '$last_name', '$email')";
    if (mysqli_query($mysqli, $insertquery)) {
        echo "Record Inserted Successfully";
    } else {
        echo "ERROR: Could not able to execute $insertquery. " .
mysqli_error($mysqli);
    }
}


// Function to display data from the database
function display($mysqli) {
    $result = mysqli_query($mysqli, "SELECT * FROM persons");
    if (mysqli_num_rows($result) > 0) {
        echo "<table border='1'>
                <tr>
                    <th>ID</th>
                    <th>First Name</th>
                    <th>Last Name</th>
                    <th>Email</th>
                </tr>";
        while ($row = mysqli_fetch_assoc($result)) {
            echo "<tr>
                    <td>{$row['id']}</td>
                    <td>{$row['first_name']}</td>
                    <td>{$row['last_name']}</td>
                    <td>{$row['email']}</td>
                  </tr>";
        }
        echo "</table>";
    } else {
        echo "No records found.";
    }
}

?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Insert and Display Data</title>
</head>
<body>
    <h1>Insert Data into Database</h1>
    <form method="POST">
        <input type="text" name="first_name" placeholder="First Name" >
        <input type="text" name="last_name" placeholder="Last Name" >
        <input type="email" name="email" placeholder="Email" >
        <button type="submit" name="insert">Submit</button>
```

```
        <button type="submit" name="display">Display</button>
    </form>

    <h2>Data from Database</h2>
    <?php
    if ($displayData) {
        display($mysqli);
    }
    ?>
</body>
</html>
```

3. Complete script for CRUD operations

```
<?php

function connect(){

    $servername = "localhost";
    $username = "root";
    $password = "";
    $dbname = "classdb";

    // Connection
    $mysqli = mysqli_connect($servername, $username, $password,$dbname);

    // Check if connection is Successful or not
    if (!$mysqli) {
        die("Connection failed: ". mysqli_connect_error());
    }
    return $mysqli;
}

//create connection variable
$mysqli = connect();

// Handle Create
if (isset($_POST['create'])) {
    $first_name = $_POST['first_name'];
    $last_name = $_POST['last_name'];
    $email = $_POST['email'];

    $insertquery = "INSERT INTO persons (first_name, last_name, email)
VALUES ('$first_name', '$last_name', '$email')";
    mysqli_query($mysqli, $insertquery);
}
```

```
// Handle Delete
if (isset($_GET['delete'])) {
    $id = $_GET['delete'];
    $deletequery = "DELETE FROM persons WHERE id=$id";
    mysqli_query($mysqli, $deletequery);
}

// Handle Update
if (isset($_POST['update'])) {
    $id = $_POST['id'];
    $first_name = $_POST['first_name'];
    $last_name = $_POST['last_name'];
    $email = $_POST['email'];

    $updatequery = "UPDATE persons SET first_name='$first_name',
last_name='$last_name', email='$email' WHERE id=$id";
    mysqli_query($mysqli, $updatequery);
}

// Handle Edit Request
$edit = false;
if (isset($_GET['edit'])) {
    $edit = true;
    $id = $_GET['edit'];
    $result = mysqli_query($mysqli, "SELECT * FROM persons WHERE id=$id");
    $row = mysqli_fetch_assoc($result);
}
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <title>CRUD Example</title>
</head>
<body>
    <h2><?php echo $edit ? 'Edit Person' : 'Add Person'; ?></h2>
    <form method="POST" action="crud.php">
        <input type="hidden" name="id" value="<?php echo $edit ?
$row['id'] : ''; ?>">
        <input type="text" name="first_name" placeholder="First Name"
required value="<?php echo $edit ? $row['first_name'] : ''; ?>">
        <input type="text" name="last_name" placeholder="Last Name"
required value="<?php echo $edit ? $row['last_name'] : ''; ?>">
        <input type="email" name="email" placeholder="Email" required
value="<?php echo $edit ? $row['email'] : ''; ?>">
        <button type="submit" name="<?php echo $edit ? 'update' :
'create'; ?>">
            <?php echo $edit ? 'Update' : 'Add'; ?>
        </button>
    </form>
```

```
    <h2>Person List</h2>
    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th><th>First Name</th><th>Last
Name</th><th>Email</th><th>Actions</th>
        </tr>
        <?php
        $result = mysqli_query($mysqli, "SELECT * FROM persons");
        while ($row = mysqli_fetch_assoc($result)) {
            echo "<tr>
                <td>{$row['id']}</td>
                <td>{$row['first_name']}</td>
                <td>{$row['last_name']}</td>
                <td>{$row['email']}</td>
                <td>
                    <a href='crud.php?edit={$row['id']}'>Edit</a> |
                    <a href='crud.php?delete={$row['id']}' onclick='return
confirm(\"Are you sure?\")'>Delete</a>
                </td>
            </tr>";
        }
        ?>
    </table>
</body>
</html>
```

Inserting multiple records at a time sent from user form

```
<?php
function connect() {
    $servername = "localhost";
    $username = "root";
    $password = "";
    $dbname = "classdb";

    $mysqli = mysqli_connect($servername, $username, $password, $dbname);
    if (!$mysqli) {
        die("Connection failed: " . mysqli_connect_error());
    }
    return $mysqli;
}

$mysqli = connect();

if (isset($_POST['create'])) {
    $multiQuery = "";
```

```
    foreach ($_POST['first_name'] as $index => $first_name) {
        $last_name = $_POST['last_name'][$index];
        $email = $_POST['email'][$index];

        if ($first_name && $last_name && $email) {
            $multiQuery .= "INSERT INTO persons (first_name, last_name,
email) VALUES ('$first_name', '$last_name', '$email');";
        }
    }

    // Execute multi-query
    if (!empty($multiQuery)) {
        if (mysqli_multi_query($mysqli, $multiQuery)) {
            echo "All records inserted successfully.";
        } else {
            echo "ERROR: " . mysqli_error($mysqli);
        }
    }
}
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <title>Insert Multiple Persons</title>
    <script>
        function addRow() {
            const table = document.getElementById('personTable');
            const newRow = table.insertRow();
            newRow.innerHTML = `
                <td><input type="text" name="first_name[]" required></td>
                <td><input type="text" name="last_name[]" required></td>
                <td><input type="email" name="email[]" required></td>
            `;
        }

        function deleteRow(button) {
            const row = button.parentNode.parentNode;
            row.parentNode.removeChild(row);
        }
    </script>
</head>
<body>
    <h2>Insert Multiple Persons</h2>
    <form method="POST" action="multiinsert.php">
        <table border="1" cellpadding="10" id="personTable">
            <tr>
                <th>First Name</th><th>Last
Name</th><th>Email</th><th>Action</th>
```

```
            </tr>
            <tr>
                <td><input type="text" name="first_name[]" required></td>
                <td><input type="text" name="last_name[]" required></td>
                <td><input type="email" name="email[]" required></td>
            </tr>
        </table>
        <br>
        <button type="button" onclick="addRow()">Add Row</button>
        <button type="submit" name="create">Submit</button>
    </form>
</body>
</html>
```

[CRUD Operations with PHP and MySQL (full tutorial for beginners) - Tony Teaches Tech]